PLAN

Introduction

The purpose of this project is to replace an IR remote controller with the use of hardware components and software in conjunction with the tm4c microcontroller. The microcontroller will use a hardcoded value for key 1 which is used as a Turn on/off switch while the other 2-16 keys will be used to record data from the IR remote controller and playback when needed using a playback command. Users will be able to communicate with the device using a common terminal interface which allows them to record data from the remote controller while also allowing them to play it back when needed.

Commands supported in the common terminal interface:
1. xmit <number>

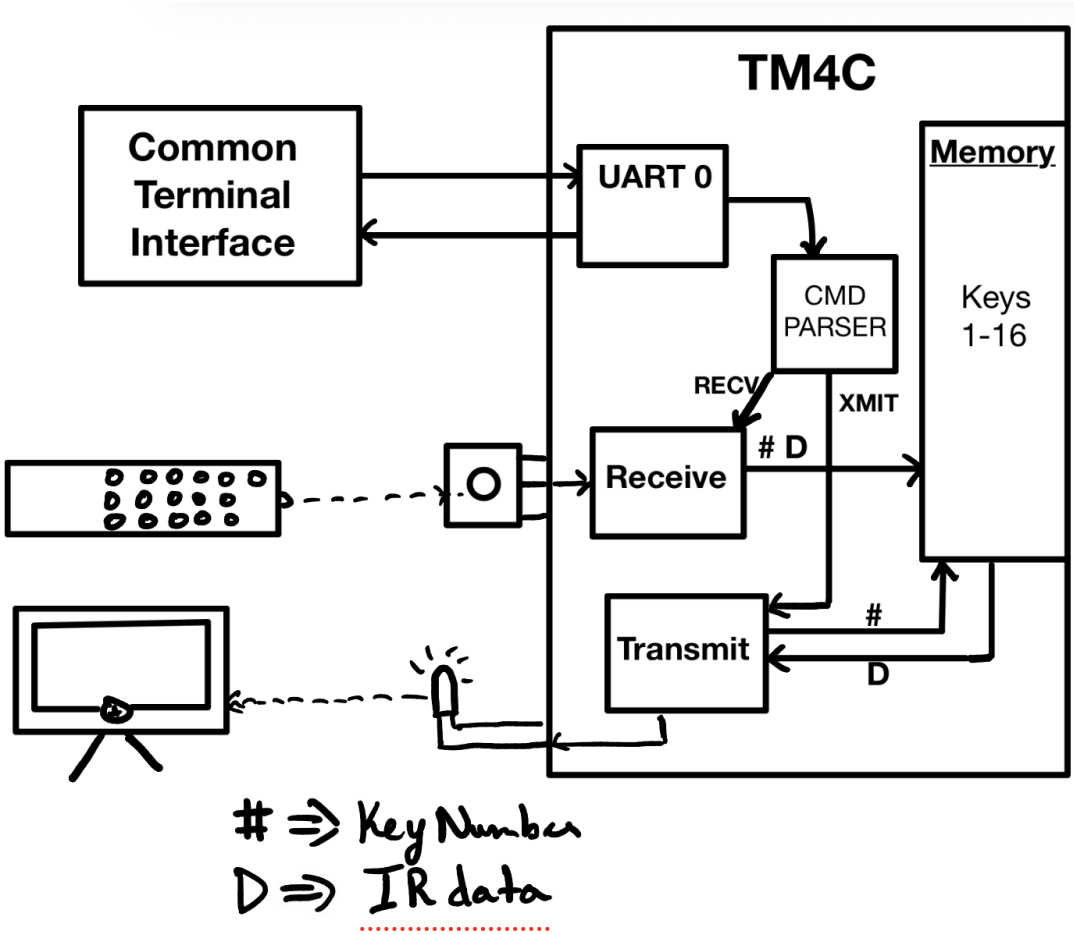
- allows the user to send data/playback via the LED

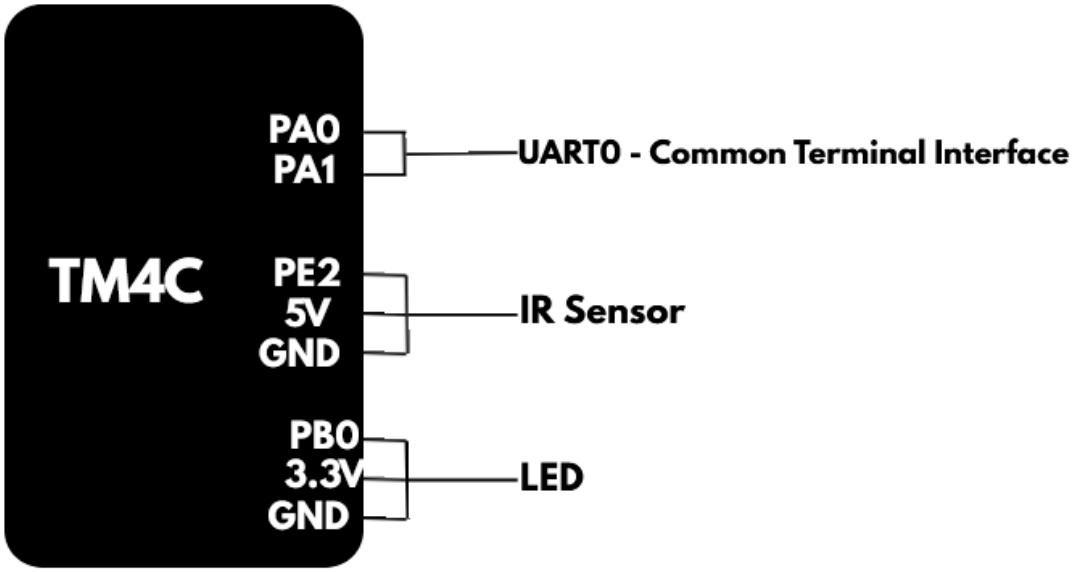
2. recv <number>

- allows the user to record data from the IR remote controller using the TSOP sensor

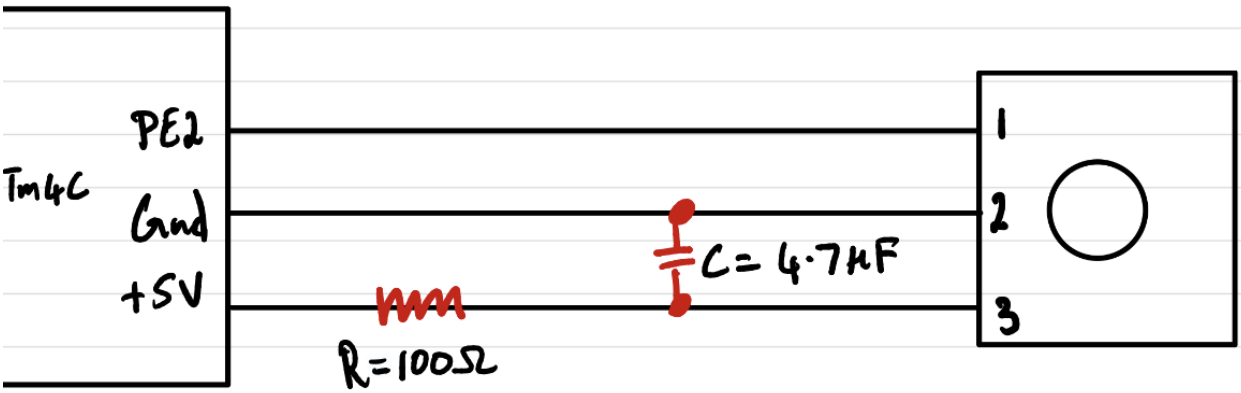
This plan will give a detailed high-level overview of the hardware approach including specific pins used in the tm4c microcontroller and any timing constraints and design requirements. It will also include details regarding the software implementation describing each thread and their parameters as well as any interrupt priorities and configuration values to meet all the requirements. Finally, it will implement a procedure to verify that the required hardware is working properly.

Block Diagram:

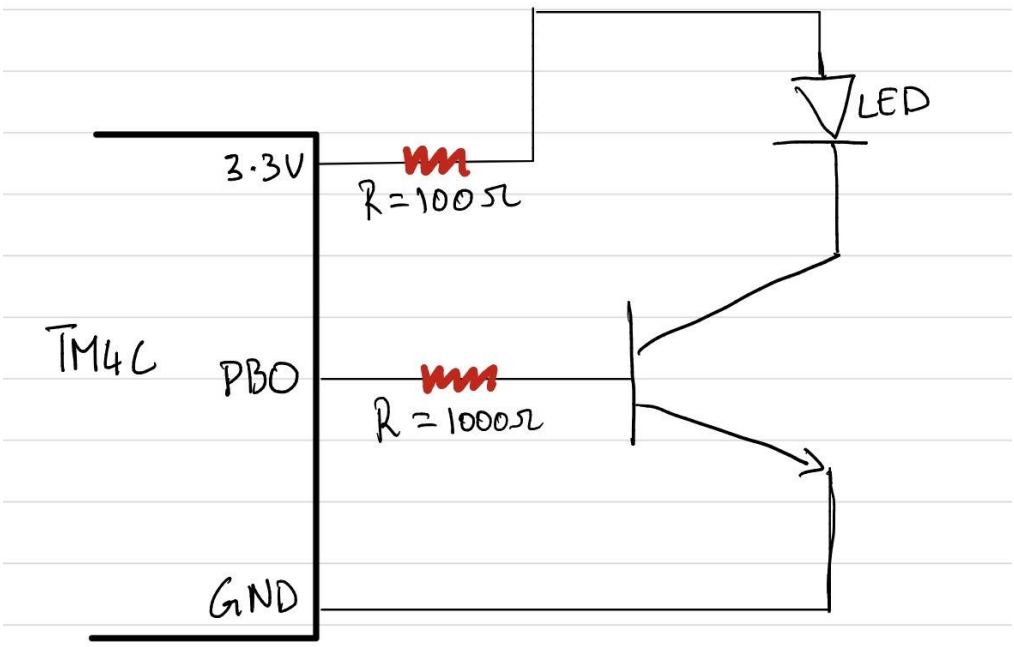




TSOP134 Hardware Setup:



IR333 Hardware setup:



Timing Constraints/requirements that exist in the design:

System Clock:

- Frequency: 40MHz
- Period: 1/40MHz = 25ns

TSOP 134:

- IR decoding:
    - Leading Code: 9000µs(carrier modulated period) + 4500µs(space period)
    - 0: 560µs (carrier modulated period) + 560µs (space period)
    - 1: 560µs (carrier modulated period) + 1690µs (space period)
    - Tail Code: 560µs (carrier modulated period)

LED:

- Frequency: 38KHz
- Period: 1/38KHz = 26.3µs = 1052 at 40MHz clock
- Half Period: 26.3/2 = 13.16µs = 526 at 40MHz clock

Uart0:

- Baud Rate: 115200
- 1 Baud: 40MHz/115200 = 347

Software Approach:

UART0: Common Terminal Interface

In this section the main idea is that we get a string from the terminal and send it over to the command parser. It has three main elements. A struct, Function to get a string from the terminal using UART0 (getsUart0) and a function to parse the string (command)

- Struct – contains the building blocks to strore a string of data
    - Char buffer[MAX_CHARS+1]
    - Uint8_t fieldcount
    - Uint8_t fieldPosition[MAX_FIELDS]
    - Char fieldType[MAX_FIELDS]
    - #define MAX_CHARS 80
    - #define MAX_FIELDS 5
- getsUart0(User_data *data) – Stores the string into the struct with the name data
- Command(USER_DATA *data) – Uses strtok to parse out the string stored in the USER_DATA struct with the name data. If the first word of the string is xmit it will send it to the transmit function along with the number that was

entered after the command. If the first word of the string is recv it will send it to the receive function along with the number that was after the command.

Timer0

- ISR – Priority 0, Toggles the PB0 pin on or off depending on its previous state every time the ISR is called.
- Periodic
- 32 bit
- Interrupt Enabled
- TIMER0_TAIL_R = 526 (Half Period)
- Enabled when the Xmit function is called and disabled after transmission is completed

XMIT

- As mentioned previously this function will enable the timer 0 which toggles PB0 on and off
- It will also contain a boolean value which during the burst period will continue to output the timer0 values of the burst cycles but when its in the space period it will continuously output 0

GPIO PORT E

- Detects both rising edge and falling edge
- ISR – Priority 1, Reads the timer1 value during each edge and subtracts it from the previous timer value and stores that value into an integer array. Enabled when recv function is called and disabled if its high for a certain period of time.
- Falling edge indicates we are in the burst period and rising edge indicated that we are in the space period this is because it is active low. Could use a not gate to invert the waveform

Struct for storing: The main job of this struct will be to store timing array values for each key.

- Max Index's for each array will be 67 because it will take 2 index's for the burst period and space period for the start, 2 index's for the burst period and the space period for each of the data bits with 32 bits in total which equates to 64 index's and one index for the stop, Giving a total of 67 index's
- Timer1 in conjunction with the number provided with recv will store each index with the exact time it was on high or low
- The only exception will be key 1 which will have a hardcoded array written for it.

TIMER1

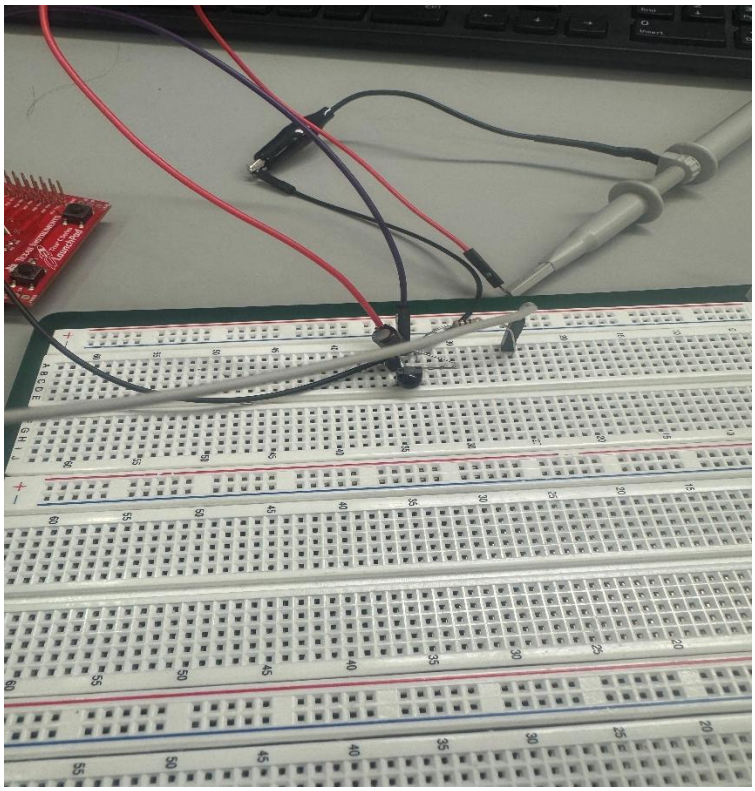
- No Interrupts Enabled
- Just keeps counting up from 0
- 64 Bit
- Periodic
- Used to get the timing values for the timing array of each keys struct

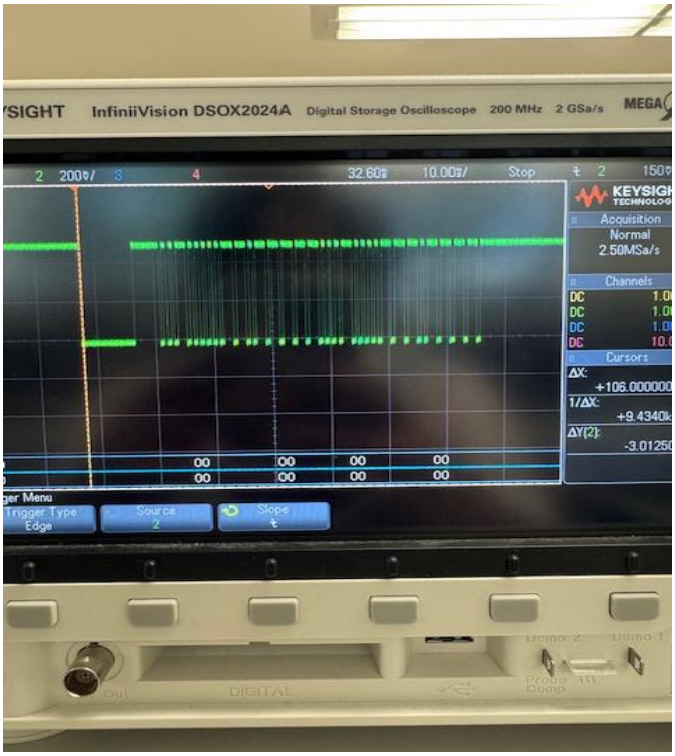
Summary: User enters either xmit or recv followed by a number. If the user enters recv followed by a number then the Port E interrupt will be enabled for both the falling and rising edge. In the ISR for Port E it will store the values of timer1 current value minus its previous value during each edge interrupt and stores that value into an array which resides within the struct for that specific key number. If the user enter xmit followed by a number then the xmit function will enable timer 0 which will cause PB0 to be on and off every 526 to representing the burst cycles, if its in the space period then it will continuously output a 0 until the next burst period. This ensures that the program is able to receive data from a remote and also replay that data using xmit after it is recorded.

Hardware Verification

TSOP Sensor:

1. Build the TSOP hardware as shown in the setup picture above on a breadboard
2. Connect the power and ground pins of the TSOP receiver to the +5V and Gnd pins on Tm4c
3. Use an oscilloscope to connect to the out pin of TSOP 134 and the other to the ground
4. Point a TV remote to the TSOP receiver and press a button
5. The Oscilloscope will show the complete frame of the data received from the TV remote including the start, data and stop bits which can be used to analyze for certain keys especially for key 1 and can also be used to verify that the hardware for the TSOP 134 is working properly



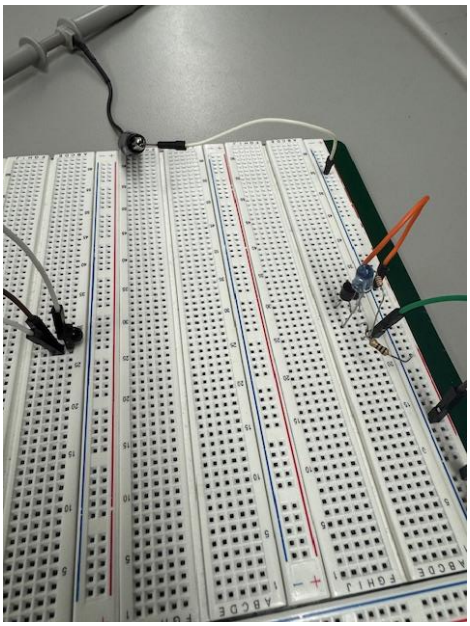


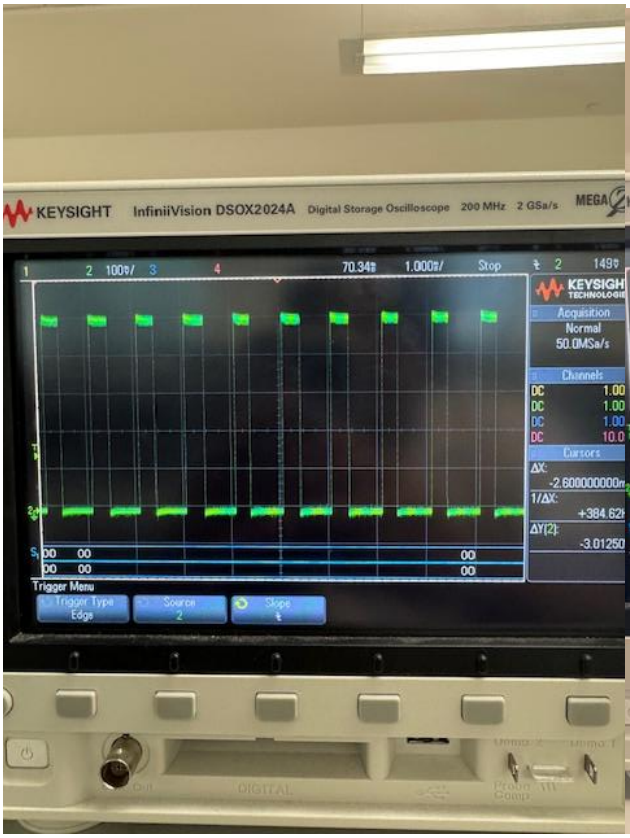
This is an example of receiving an on/off button press from the remote

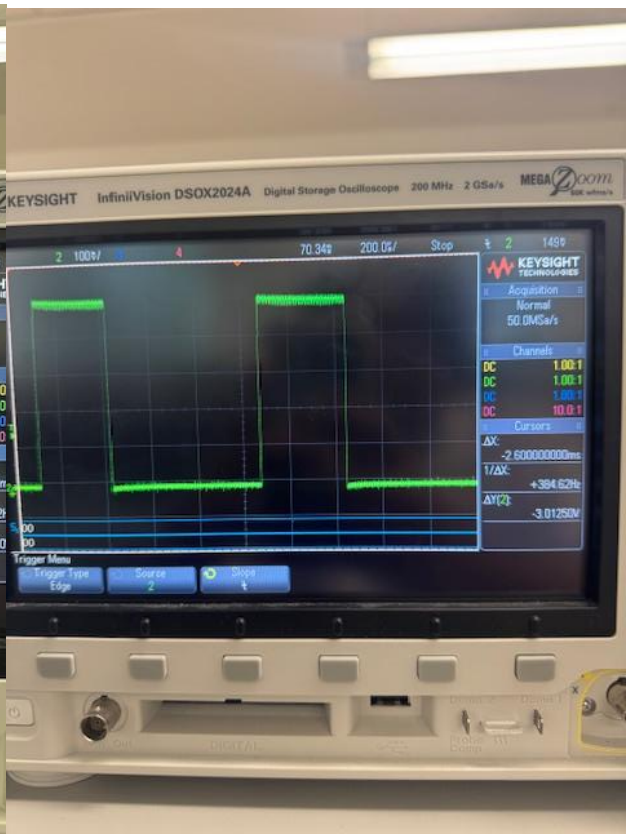
LED:

1. Build the LED hardware as shown in the setup picture above on a breadboard
2. Connect the 3.3V to the 100 ohm resistor going into the led, the ground to one of the ground pins on tm4c and the PB0 pin to the base of the transistor
3. Manually output a message using the software to PB0
4. Check that the LED is outputting the given message by using the TSOP 134 to verify that the hardware is setup properly

```c
#include "clock.h"
#include "tm4c123gh6pm.h"
#include <stdint.h>

#define TX_MASK 0x01

int main(void)
{
    initSystemClockTo40Mhz();

    SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R1;
    _delay_cycles(3);

    GPIO_PORTB_DEN_R |= TX_MASK;
    GPIO_PORTB_DIR_R |= TX_MASK;
    GPIO_PORTB_DR2R_R |= TX_MASK;
    volatile uint32_t i;
    volatile uint32_t j;
    while(1)
    {
        for(i = 0; i < 24; i++)
        {
            GPIO_PORTB_DATA_R |= TX_MASK;
            _delay_cycles(526);
            GPIO_PORTB_DATA_R &= ~TX_MASK;
            _delay_cycles(526);
        }
        for(j = 0; i < 50; i++)
        {
            GPIO_PORTB_DATA_R &= ~TX_MASK;
            _delay_cycles(526);
        }
    }
    return 0;
}

```







Software code being outputted by the LED and caught by the receiver on the oscilloscope.